

# **UNIDAD 8**

## **DESARROLLO DE INTERFACES GRÁFICAS**

### **EJERCICIOS ADICIONALES**

**PROGRAMACIÓN  
CFGS DAW**

Autor: José Miguel Blázquez  
2023/2024

## Licencia



**Reconocimiento - NoComercial - CompartirIgual (by-nc-sa):** No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

# UD8. DESARROLLO DE INTERFACES GRÁFICAS

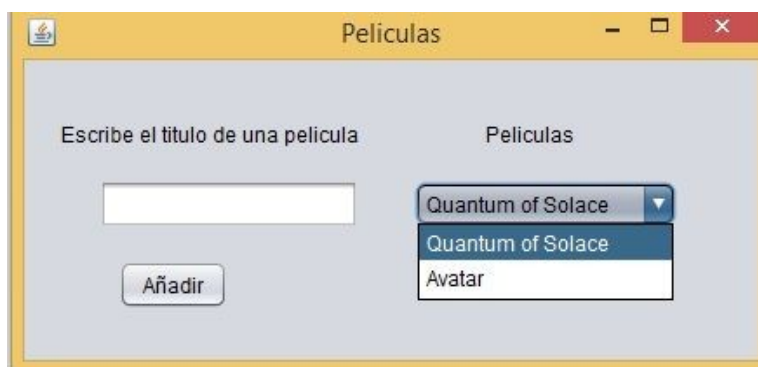
## EJERCICIOS

Implementa las aplicaciones gráficas siguiendo las especificaciones de diseño y funcionalidad descritas en cada ejercicio. Ten en cuenta lo siguiente:

- **Los aspectos del diseño que no se describan quedan a tu elección.** En todo caso, intenta que la interfaz sea sencilla de utilizar para el usuario, debes diseñar pensando en su usabilidad.
- **Renombra todos los componentes para que se entienda su uso.** En lugar de `TextField1`, `Button1` y `Label1` es mejor por ejemplo `TextFieldEdad`, `ButtonSuma` y `LabelResultado`. También puedes utilizar abreviaturas: `txtEdad`, `btnSuma` y `lblRes`.
- **Comprueba la información introducida por el usuario** y avísale en caso de error.
- **Maneja las posibles excepciones** que puedan producirse.
- **Puedes implementar clases adicionales** si lo consideras oportuno.
- **La ventana principal debe tener un nombre corto y descriptivo**, no lo dejes en blanco.

### Ejercicio 1 – Lista de películas modificable

Crea una lista de películas en un Combobox. Allí tendrás que almacenar las películas que el usuario escriba en un campo de texto. Al pulsar el botón “**Añadir**” la película escrita se introducirá en el ComboBox y podrá ser seleccionada. Cuando se seleccione una película, mostrarás una ventana emergente informativa indicando “Has seleccionado la película: <nombre película>”.



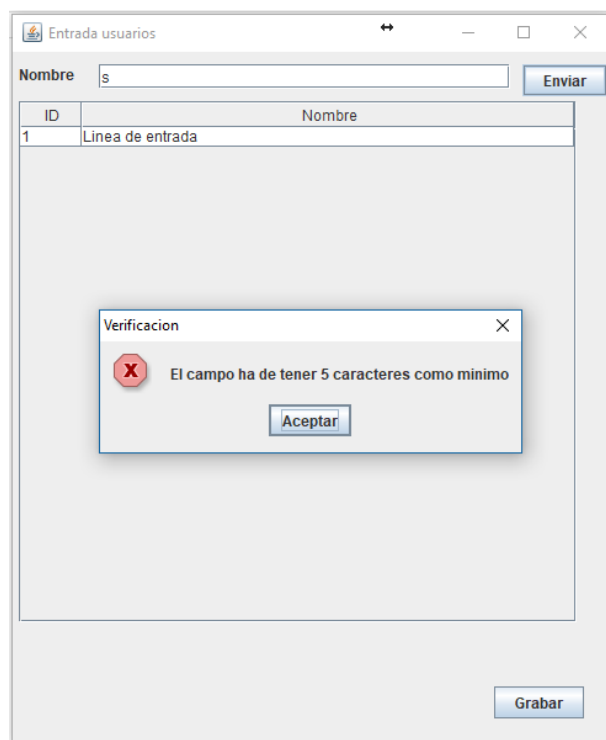
### Ejercicio 2 – Listado de usuarios participantes

Necesitamos grabar los nombres de nuestros usuarios, para ello, prepararemos una pantalla (con JFrame) que solicite el nombre, y con un botón, que al pulsarlo, lo añada a la tabla, junto con un identificador que empezará en 1 y se incrementará automáticamente.

Si al pulsas en “Enviar” el nombre está en blanco, o tiene una longitud menor de 5 caracteres, mostraremos una ventana emergente de error y se volverá a pedir.

\* **Ampliación 1:** añade la posibilidad de borrar un usuario cualquiera que selecciones de la lista (luego se tendrán que recalcular los ID de todos).

\* **Ampliación 2:** almacena los usuarios en disco y recuperalos mediante 2 botones. Ambas opciones solicitarán el fichero mediante un JFileChooser.



### Ejercicio 3 – Gestión de usuarios

Se requiere desarrollar una aplicación que permita la gestión de personas de un sistema a través de un formulario en Swing, las cuales se almacenarán en un ArrayList. Las personas pueden ser “usuario” o “administrador”. Atributos comunes a ambos roles: DNI, nombre, apellidos, departamento, fecha de incorporación, nombre de usuario, contraseña. Atributos de admin: nivel de acceso (lista desplegable con opciones: Total ó Parcial, por defecto Parcial). Atributos de usuario: cuota de espacio en disco asignada (número entero en MB).

La aplicación debe permitir realizar las siguientes operaciones:

1. **Alta de Usuario/Administrador:** el formulario debe permitir introducir los datos comunes de una persona, así como seleccionar si se está creando un usuario o un administrador y rellenar los campos adicionales. La contraseña se introducirá 2 veces (2 campos) y no se tendrá que ver (mostrar \*\*\*\*).
2. **Consulta de Usuarios/Administradores:** Se debe poder visualizar en una tabla todos los perfiles, bien de tipo usuario o de tipo administrador (elegir opción).
3. **Modificación de Usuario/Administrador:** Se debe permitir modificar los datos de un usuario o administrador, para ello se consultará por nombre de usuario.
4. **Borrado de Usuario/Administrador:** Se debe poder eliminar un usuario o administrador seleccionando por nombre de usuario.

Los usuarios y administradores deberán ser almacenados en un ArrayList que permita su gestión durante la ejecución de la aplicación. La interfaz gráfica debe ser intuitiva y fácil de usar para el usuario final.

## Ejercicio 4 – TPV Hamburguesería

Implementa una aplicación gráfica que simule una aplicación de pedidos de menús de una hamburguesería. La aplicación permitirá configurar un solo menú cada vez donde el usuario (que será el dependiente del restaurante) marcará las opciones que el cliente le indique, y la aplicación calculará el precio total a cobrar. El usuario podrá elegir entre estas opciones (elección obligatoria): tipo de hamburguesa, pan, patatas y bebida. También hay elementos opcionales/extra que serán tenidos en cuenta de forma adicional pero no obligarán al usuario a seleccionarlos.

### PARTE 1 (Interfaz y funcionalidad básica)

El precio del menú básico es de 8 euros pero algunas opciones tienen un coste adicional que se tendrá que sumar. En este coste se incluirán las opciones básicas del menú. Sólo se incrementará el precio si se selecciona una opción adicional.

Las opciones obligatorias a elegir son (entre paréntesis se indica si suman el precio o no. Las opciones sin precio están incluidas en el precio estándar del menú):

- Hamburguesa a elegir: pollo, cerdo, ternera (+1€) o vegana (+1,25€).
- Pan a elegir: normal, integral o centeno (+0,25€).
- Patatas a elegir: fritas, gajo (+0,5€).
- Bebida a elegir: refresco de cola, refresco de naranja, refresco de limón, agua o cerveza.

Las opciones extra/adicionales (se pueden marcar o no, de forma individual) son:

- Hamburguesa doble (+2 €).
- Extra de queso (+0,50 €).
- Extra de patatas (+1 €).
- Salsas: ketchup, barbacoa y mostaza. Pueden pedirse varios paquetitos de cada una (+0,25€ la unidad).
- Un elemento cualquiera adicional (permitir que se introduzca manualmente un precio en euros en ese caso).

Además, tras todo lo anterior también se tendrá que seleccionar:

- Reparto a domicilio (+1€) o recogida en el local [recogida en local por defecto].
- Aplicar descuento (-10%) por cliente habitual [opción optativa].

Cuando esté todo seleccionado, mediante la pulsación de un botón, la aplicación tendrá que calcular y mostrar el resultado desglosado en varios JTextField(s):

- el precio **total bruto** del menú con las opciones elegidas en la pantalla, en bruto
- el valor del **IVA** en € (4% sobre el total anterior)
- el precio de **venta** al público (tras sumar el IVA al valor total bruto).

También deberá mostrar un listado en un (JTextArea) con la configuración elegida del menú.

**Aspectos** a tener en cuenta:

- La aplicación debe tener una ventana de tamaño fijo no redimensionable, y debe tener un título o nombre.

- Debe tener un botón adicional para hacer un nuevo pedido, que inicializará las opciones para volver a configurar un pedido desde cero.
- Controla las posibles excepciones que puedan haber.
- Si necesitas informar al usuario de alguna situación, hazlo con una ventana emergente apropiada.
- Cuida el diseño: orden, formato visual y facilidad de uso para el usuario.

## **PARTE 2 (Ampliación con uso de objetos)**

Ahora vas a añadir la posibilidad de almacenar los pedidos en formato "objeto" en una estructura, para ello:

1. Crea la clase "Pedido", que tenga los atributos necesarios para poder almacenar dentro toda la información que indique el usuario en la configuración del menú (así como un campo que sirva para identificar el pedido, del estilo "idpedido" o "numpedido" que tendrá que ser diferente para cada instancia).
2. Modifica la interfaz para que tenga un botón adicional para "Almacenar pedido". El botón creará un nuevo objeto de tipo "Pedido" pasándole al constructor todos los datos de la interfaz, dicho objeto será almacenado en un ArrayList de tipo "Pedido".

## **AMPLIACIONES EXTRA RECOMENDADAS**

- Añade a la interfaz la posibilidad de consultar un pedido a través de la introducción del número o id del pedido. Al hacerlo, si este existe, mostrará la configuración en la interfaz pero NO permitirá su modificación (bloqueando todos los controles).
- Añade un botón para calcular el importe total recaudado con los pedidos que hay almacenados en el ArrayList, el resultado total lo mostrará en una ventana emergente.